



UNIVERSIDAD DE CONCEPCIÓN

FACULTAD DE INGENIERÍA

DEPARTAMENTO DE INGENIERÍA INFORMÁTICA

Informe tarea

Escape de la torre

Nombre:

Leonardo Guerrero

Asignatura: Inteligencia Artificial

Profesor: Julio Godoy

Fecha: 28 de septiembre de 2026

Índice

1. Introducción	2
2. Descripción del problema y cumplimiento del enunciado	3
2.1. Restricciones consideradas	3
2.2. Correspondencia entre el enunciado y el código	3
3. Metodología de la solución	4
3.1. Modelo formal del entorno	4
3.2. Congestión, capacidad y espera	4
3.3. Propagación del fuego	4
3.4. Protocolo experimental	5
3.5. Definición de las métricas	5
4. Algoritmos implementados	6
4.1. Búsqueda no informada	6
4.1.1. Búsqueda en anchura (BFS)	6
4.1.2. Búsqueda en profundidad (DFS)	6
4.2. Búsqueda informada	6
4.2.1. A*	6
4.2.2. IDA*	7
4.3. Optimización bioinspirada: algoritmo genético	7
5. Implementación de interfaz y herramientas auxiliares	8
6. Resultados experimentales	8
6.1. Resultados con propagación cada tres turnos ($k = 3$)	8
6.2. Resultados con propagación cada cuatro turnos ($k = 4$)	9
6.3. Visualización de resultados	10
6.4. Discusión	11
7. Limitaciones y mejoras	11
8. Conclusión	12
9. Declaración de uso de IA generativa y procedencia	12

1. Introducción

La planificación de trayectorias en ambientes dinámicos constituye un problema central de la Inteligencia Artificial. En una evacuación, encontrar geoméricamente un camino hasta la salida no es suficiente: los agentes deben desplazarse por una infraestructura parcialmente bloqueada, evitar zonas que se incendian, resolver conflictos de ocupación y reaccionar ante decisiones tomadas por otros agentes. Por ello, el problema combina búsqueda en grafos, planificación bajo incertidumbre y coordinación descentralizada.

El escenario representa un edificio mediante una grilla bidimensional por piso. En cada piso existe una única salida de evacuación y un grupo de personas liderado por Sebastián. El incendio avanza a intervalos discretos, las celdas consumidas se vuelven intransitables y los agentes que no alcanzan la salida antes de ser alcanzados por las llamas se consideran bajas. Además, los pasillos tienen capacidad finita: cuando demasiadas personas intentan utilizar una misma celda, se produce congestión y algunos agentes deben esperar.

El objetivo del trabajo es implementar y comparar los tres paradigmas exigidos:

1. **Búsqueda no informada:** BFS y DFS.
2. **Búsqueda informada:** A* e IDA*, guiados por una heurística admisible.
3. **Optimización bioinspirada:** un algoritmo genético para planificar secuencias de acciones.

La comparación se realizó sobre tres mapas de 50 por 50 celdas: Cuello de Botella, Laberinto Corporativo y Dispersión Abierta. Se analizaron dos frecuencias de propagación del fuego, $k = 3$ y $k = 4$, mediante 200 simulaciones por combinación. Las métricas principales son la tasa de supervivencia y los estadísticos descriptivos de los turnos de despeje.

2. Descripción del problema y cumplimiento del enunciado

2.1. Restricciones consideradas

El sistema satisface las restricciones principales del problema:

- La representación espacial es una grilla regular de filas y columnas.
- Los movimientos planificados son ortogonales: arriba, abajo, izquierda y derecha.
- El algoritmo genético incorpora también la acción de esperar $(0, 0)$.
- Cada celda posee una capacidad física finita y un costo dependiente de su ocupación.
- El fuego se propaga cada k turnos y convierte las celdas alcanzadas en obstáculos permanentes.
- Los agentes pueden replanificar cuando su ruta se vuelve inválida.
- La salida es única por mapa y la evacuación se registra cuando un agente llega a ella.

El análisis se centra en la supervivencia y en los turnos de simulación. No se utiliza el tiempo de CPU como métrica principal, por lo que las tablas y figuras no permiten comparar directamente el tiempo de ejecución computacional de los algoritmos.

2.2. Correspondencia entre el enunciado y el código

La clase **GestorDeEventos** funciona como motor de la simulación. En cada turno incrementa el reloj, mezcla aleatoriamente el orden de los agentes activos, solicita a cada agente que se mueva y, cuando corresponde, propaga el fuego.

La clase **Agente** mantiene la posición actual, la posición anterior, el estado de evacuación, el estado de fallecimiento, la ruta planeada y el estado necesario para replanificar. Antes de moverse verifica si está en fuego o en la salida. Luego valida la ruta almacenada; si el siguiente paso dejó de ser transitable, vuelve a buscar. Si una celda se encuentra saturada según la restricción de capacidad, el agente espera.

La clase **Casilla** representa el tipo de celda y la lista de agentes presentes. El lector interpreta los símbolos definidos por el formato del mapa, mientras que el utilitario de conversión permite transformar una imagen PNG en un archivo textual utilizable por el simulador.

3. Metodología de la solución

3.1. Modelo formal del entorno

Sea $M \in \mathbb{R}^{F \times C}$ una grilla de F filas y C columnas. Cada posición contiene una casilla cuyo tipo puede ser camino, muro o fuego; la salida se almacena como una coordenada objetivo S . Los estados muro y fuego son intransitables.

El grafo implícito del mapa se define como

$$G = (V, E), \quad (1)$$

donde V contiene las celdas transitables y $(u, v) \in E$ si u y v son vecinas ortogonales. Este grafo cambia durante la simulación porque una celda de camino puede transformarse irreversiblemente en fuego.

3.2. Congestión, capacidad y espera

Para una celda destino ocupada por n agentes, se utiliza la función cuadrática:

$$C(n) = 1 + n^2. \quad (2)$$

La capacidad física implementada es:

$$k_{\text{cap}} = 6. \quad (3)$$

La capacidad y el costo cumplen funciones relacionadas, pero no idénticas. La capacidad impone una restricción durante el movimiento, mientras que el costo cuadrático permite que las búsquedas que consideran costos prefieran, cuando existen alternativas, celdas menos ocupadas.

En el caso del algoritmo genético, el mecanismo de respaldo basado en memoria evita preferentemente celdas que ya han sido visitadas muchas veces, pero no convierte toda celda ocupada en una posición prohibida. La memoria registra las visitas de manera acotada y se consulta solamente durante la selección del movimiento de respaldo del algoritmo genético.

3.3. Propagación del fuego

El origen del fuego se selecciona de manera completamente aleatoria, aunque se evita que aparezca en las últimas filas, dado que en los mapas la salida siempre se encuentra en el borde inferior. Esto permite evitar que el fuego aparezca demasiado cerca de la salida y afecte de manera directa el análisis de los resultados. Por otro lado, tampoco se seleccionan como origen las casillas ocupadas por los agentes ni sus casillas vecinas, por la misma razón.

La propagación ordinaria del fuego utiliza probabilidad

$$p = 0.75 \quad (4)$$

y ocurre cada k turnos.

En las ejecuciones del benchmark por consola, cuando el fuego encuentra un muro puede intentar propagarse al otro lado. Si el grosor del muro es d , la probabilidad aplicada es

$$P_{\text{pared}}(d) = p_{\text{pared}}^d, \quad (5)$$

con el valor predeterminado

$$p_{\text{pared}} = 0.25. \quad (6)$$

Por lo tanto, el espesor del muro modifica la probabilidad efectiva de atravesarlo. Este parámetro debe quedar registrado junto con el resto de las condiciones experimentales para facilitar la reproducción de los resultados.

Las celdas alcanzadas por el fuego dejan de ser transitables y los agentes presentes en ellas son registrados como bajas. La salida no se incendia.

3.4. Protocolo experimental

Se realizaron 200 simulaciones independientes por cada combinación de mapa, algoritmo y frecuencia de propagación. Esto permite superar tanto el mínimo de 80 iteraciones como la referencia de 200 ejecuciones indicada para el experimento.

No se fijó una semilla global de aleatoriedad. En consecuencia, pueden variar entre ejecuciones el foco inicial del incendio, el orden de actuación de los agentes y la expansión del fuego. Para facilitar la comparación entre futuras ejecuciones, también es necesario registrar los parámetros completos del fuego, incluida la probabilidad de atravesar paredes.

3.5. Definición de las métricas

La tasa de supervivencia calculada por el benchmark es

$$\text{Supervivencia} = \frac{N_{\text{evacuados}}}{N_{\text{total}}} \times 100. \quad (7)$$

En consecuencia, un agente se contabiliza como superviviente cuando alcanza la salida.

El tiempo de despeje utilizado por la implementación corresponde al turno de evacuación más tardío entre los agentes que sí evacuaron. Si una ejecución no registra evacuaciones, el tiempo de despeje no queda definido. Por esta razón, el benchmark informa adicionalmente el número de corridas con tiempo definido.

Las estadísticas de turnos reportadas corresponden a la media, desviación estándar, mínimo y máximo de las corridas que producen un tiempo definido. Esta métrica debe distinguirse del tiempo de ejecución computacional: los turnos representan tiempo dentro de la simulación, no segundos de CPU.

4. Algoritmos implementados

4.1. Búsqueda no informada

4.1.1. Búsqueda en anchura (BFS)

BFS comienza en la posición del agente y explora los estados por niveles de profundidad. La cola FIFO garantiza que los estados se procesen en orden no decreciente de profundidad. El registro de padres permite reconstruir la trayectoria al alcanzar la salida.

En un grafo no ponderado, BFS es completo y encuentra una ruta con el menor número de movimientos si existe. No obstante, no incorpora directamente la congestión ni la evolución futura del fuego en su función de prioridad.

4.1.2. Búsqueda en profundidad (DFS)

DFS utiliza una pila LIFO y profundiza en una rama antes de retroceder. El conjunto de visitados evita ciclos infinitos, pero no evita que la primera rama elegida sea poco conveniente.

DFS no garantiza una trayectoria mínima y puede producir recorridos largos. En este proyecto, el orden de vecinos influye en la trayectoria obtenida y una trayectoria prolongada puede aumentar la exposición al fuego y la congestión.

4.2. Búsqueda informada

Ambos algoritmos informados reciben una función heurística basada en distancia Manhattan:

$$h(a, b) = |f_a - f_b| + |c_a - c_b|. \quad (8)$$

Como los movimientos son ortogonales, la distancia Manhattan no supera el número mínimo de pasos restantes en un mapa sin obstáculos. Además, la función de congestión satisface $C(n) \geq 1$, por lo que Manhattan constituye una cota inferior del costo de desplazamiento cuando los costos de las celdas son no negativos.

La heurística no predice la propagación futura del incendio ni las decisiones de los demás agentes.

4.2.1. A*

A* combina el costo acumulado y la estimación restante:

$$f(n) = g(n) + h(n). \quad (9)$$

Para un vecino n' se calcula

$$g(n') = g(n) + C(n'). \quad (10)$$

La implementación actualiza el costo conocido y el padre cuando encuentra una alternativa de menor costo, utilizando una cola de prioridad. Al alcanzar la salida se reconstruye la secuencia de coordenadas.

En la simulación, A* puede ejecutarse nuevamente cuando la ruta almacenada deja de ser válida. Por tanto, la planificación es reactiva y utiliza el estado de ocupación disponible en el momento de cada búsqueda.

4.2.2. IDA*

IDA* combina la exploración en profundidad con umbrales sucesivos de A*. Inicialmente utiliza un umbral basado en $h(\text{inicio})$. La búsqueda recursiva poda ramas cuyo valor $f(n)$ supera el umbral y utiliza el menor valor excedido para definir el siguiente umbral.

La implementación conserva, dentro de cada búsqueda, el mejor costo conocido para los estados visitados y evita repetir posiciones ya presentes en la ruta actual.

Para limitar búsquedas prolongadas, cada llamada a IDA* tiene un presupuesto máximo de 500 nodos expandidos. La implementación también registra cuándo se alcanza este límite. Esta restricción controla el costo de una llamada, pero puede impedir encontrar una ruta si la exploración necesaria supera el presupuesto.

4.3. Optimización bioinspirada: algoritmo genético

El algoritmo genético representa una trayectoria como un cromosoma de acciones. Cada gen corresponde a un desplazamiento ortogonal o a la acción de esperar.

La población inicial utiliza información de distancia Manhattan para favorecer, con cierta probabilidad, acciones que acerquen al objetivo, combinándolas con acciones aleatorias. Cada individuo es evaluado mediante una función de fitness que considera costos de congestión y penalizaciones asociadas a movimientos inválidos o a no alcanzar la salida.

Posteriormente se seleccionan individuos, se realiza cruce de un punto y se aplica mutación. La mutación sustituye el gen seleccionado por una acción distinta, evitando mutaciones que no alteren el cromosoma. Además, se conserva la mejor ruta factible encontrada durante la evolución.

Cuando el algoritmo genético no obtiene una ruta factible, el agente utiliza un movimiento de respaldo guiado por una memoria dispersa de las celdas visitadas. El conteo de visitas por celda está acotado y, al elegir un vecino, se priorizan los menos visitados; los empates favorecen evitar el retroceso inmediato y acercarse a la salida. Este mecanismo se habilita solamente para el algoritmo genético.

El algoritmo genético no garantiza encontrar una trayectoria óptima ni una solución en un tiempo determinado. Además, su función de evaluación constituye una aproximación del entorno, ya que durante la evolución de un cromosoma no se reproduce completamente la evolución futura del fuego y la interacción simultánea entre agentes.

5. Implementación de interfaz y herramientas auxiliares

La interfaz opcional se implementó con Pygame y permite visualizar el estado de la simulación. El benchmark de consola es el mecanismo utilizado para generar las estadísticas presentadas en este informe.

El conversor de PNG a texto utiliza colores codificados para representar camino, muro, agente y salida. Posteriormente, el lector de mapas transforma el archivo textual en objetos del dominio. Esta separación permite modificar la topología sin alterar directamente los algoritmos de búsqueda.

6. Resultados experimentales

6.1. Resultados con propagación cada tres turnos ($k = 3$)

Cuadro 1: Resultados del benchmark con $k = 3$. “Def.” corresponde a corridas con tiempo de despeje definido.

Mapa	Alg.	Sup. (%)	Media	DE	Mín.	Máx.
Cuello	BFS	78.64	64.17	8.18	30	83
	DFS	18.91	100.11	42.18	37	196
	A*	75.81	64.15	9.19	33	84
	IDA*	45.51	86.02	29.26	31	166
	Genético	18.89	65.37	29.19	32	197
Laberinto	BFS	81.34	64.53	10.85	24	118
	DFS	16.84	115.76	35.86	40	194
	A*	78.93	63.38	12.40	25	103
	IDA*	68.91	76.67	25.00	24	171
	Genético	26.46	64.03	29.63	29	204
Dispersión	BFS	87.61	65.54	6.75	27	86
	DFS	2.25	118.34	26.82	70	200
	A*	86.52	64.92	9.95	16	87
	IDA*	85.76	64.77	6.32	27	68
	Genético	70.58	76.89	17.71	34	183

Corridas con tiempo definido: Cuello: 199/200, 195/200, 198/200, 200/200 y 192/200; Laberinto: 200/200, 172/200, 200/200, 200/200 y 195/200; Dispersión: 200/200 en todos los algoritmos salvo DFS, con 139/200.

Con $k = 3$, las tasas de supervivencia de BFS y A* se mantienen entre 75.81 % y 87.61 % en los tres mapas. En Cuello de Botella, BFS alcanza 78.64 % y A* 75.81 %, con medias de 64.17 y 64.15 turnos, respectivamente. En Laberinto Corporativo, BFS registra 81.34 % y A* 78.93 %, mientras que sus medias son 64.53 y 63.38 turnos.

En Dispersión Abierta, BFS y A* alcanzan 87.61 % y 86.52 % de supervivencia, respectivamente. IDA* también presenta un resultado cercano, con 85.76 % de supervivencia y 64.77 turnos de media.

DFS presenta tasas de supervivencia considerablemente menores, especialmente en Dispersión Abierta, donde alcanza 2.25 %. Sus medias de turnos también son mayores en los tres mapas.

El algoritmo genético muestra un comportamiento dependiente de la topología. Su supervivencia alcanza 70.58 % en Dispersión Abierta, pero es de 18.89 % en Cuello de Botella y 26.46 % en Laberinto Corporativo.

6.2. Resultados con propagación cada cuatro turnos ($k = 4$)

Cuadro 2: Resultados del benchmark con $k = 4$. “Def.” corresponde a corridas con tiempo de despeje definido.

Mapa	Alg.	Sup. (%)	Media	DE	Mín.	Máx.
Cuello	BFS	86.18	68.56	9.34	34	140
	DFS	29.46	130.18	56.03	37	256
	A*	83.88	68.18	8.46	40	123
	IDA*	54.91	111.96	36.43	31	203
	Genético	21.18	71.41	35.77	40	240
Laberinto	BFS	87.10	67.73	9.93	35	142
	DFS	25.29	139.26	54.24	43	264
	A*	87.53	68.58	9.85	31	112
	IDA*	74.78	93.17	35.21	31	220
	Genético	27.80	68.00	35.71	32	253
Dispersión	BFS	90.33	66.23	5.16	38	74
	DFS	5.17	143.84	49.35	66	267
	A*	88.60	66.67	7.42	22	84
	IDA*	89.91	67.44	7.53	40	126
	Genético	76.12	79.44	20.74	24	251

Corridas con tiempo definido: Cuello: 200/200, 198/200, 199/200, 200/200 y 197/200; Laberinto: 200/200, 186/200, 200/200, 200/200 y 200/200; Dispersión: 200/200, 173/200, 200/200, 200/200 y 200/200.

Con $k = 4$, las tasas de supervivencia aumentan respecto de $k = 3$ en la mayoría de las configuraciones. En Cuello de Botella, BFS alcanza 86.18 % y A* 83.88 %. En Laberinto Corporativo, BFS y A* presentan 87.10 % y 87.53 %, respectivamente. En Dispersión Abierta, BFS llega a 90.33 %, A* a 88.60 % e IDA* a 89.91 %.

El algoritmo genético también mejora en el mapa abierto, alcanzando 76.12 % de supervivencia. En cambio, sus resultados permanecen considerablemente menores en Cuello de Botella y Laberinto Corporativo.

Las diferencias entre $k = 3$ y $k = 4$ deben interpretarse junto con la aleatoriedad del experimento. Por ejemplo, A* en Cuello de Botella pasa de 75.81 % a 83.88 %, mientras que en otros casos la variación es menor. Las 200 corridas permiten observar la distribución, pero no eliminan por sí solas la variabilidad del proceso estocástico.

6.3. Visualización de resultados

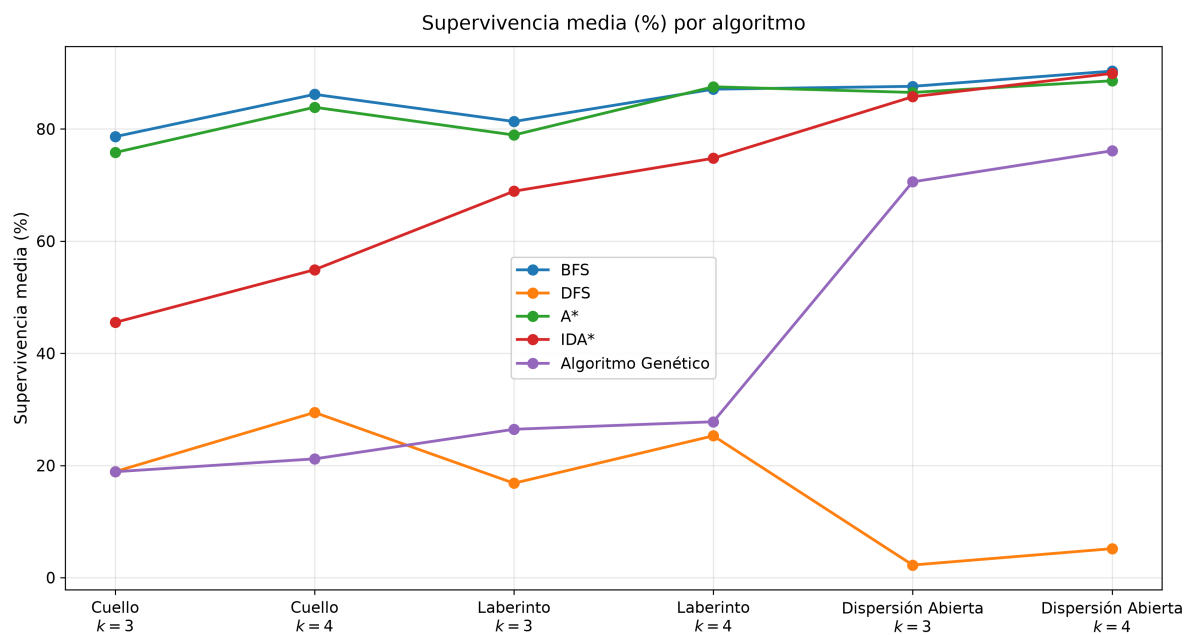


Figura 1: Tasa de supervivencia para los seis escenarios experimentales.

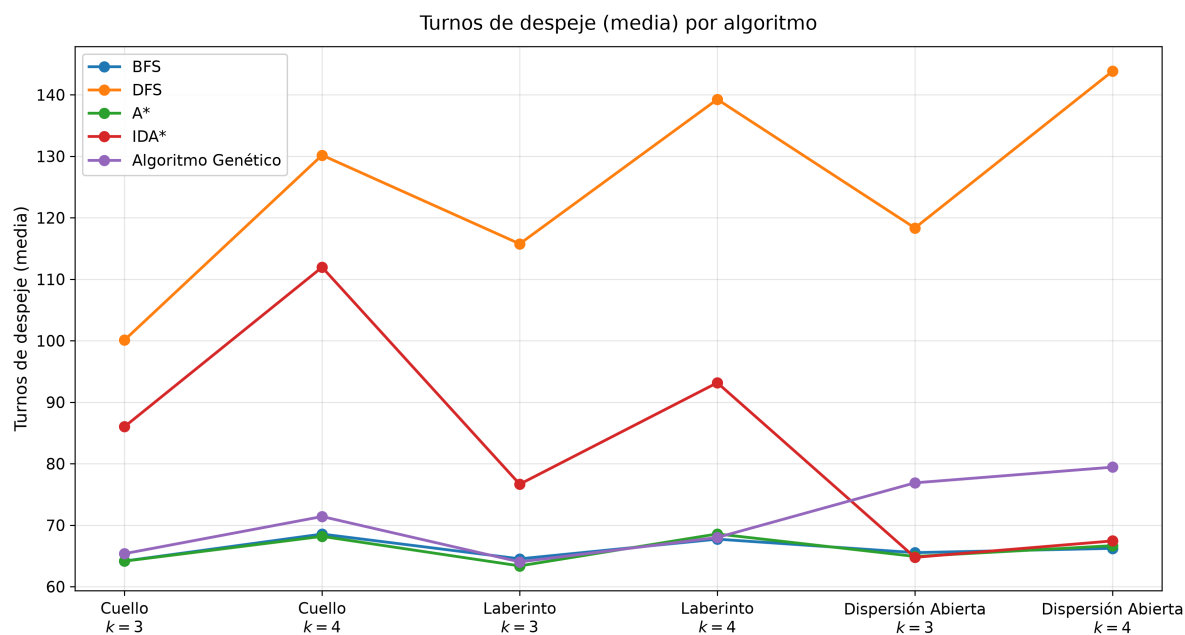


Figura 2: Media de turnos de despeje para los seis escenarios experimentales.

6.4. Discusión

Los resultados muestran diferencias asociadas tanto al algoritmo como a la topología del mapa. BFS y A* mantienen tasas de supervivencia elevadas en las seis configuraciones, mientras que DFS presenta valores considerablemente inferiores. En el caso de DFS, se observa que en Dispersión Abierta la tasa de supervivencia es especialmente baja, mientras que en Cuello de Botella es comparativamente mayor. Una posible explicación es que, en la topología abierta, DFS puede recorrer durante más tiempo caminos poco convenientes antes de alcanzar la salida, aumentando su exposición al fuego.

IDA* presenta un comportamiento más variable. En Dispersión Abierta obtiene 85.76 % con $k = 3$ y 89.91 % con $k = 4$, pero sus tasas disminuyen a 45.51 % y 54.91 % en Cuello de Botella. El límite de 500 nodos puede impedir que algunas búsquedas encuentren una ruta antes de consumir su presupuesto.

El algoritmo genético presenta una dependencia particularmente visible de la topología. Su tasa de supervivencia aumenta de 18.89 % a 70.58 % para $k = 3$ al pasar de Cuello de Botella a Dispersión Abierta, y de 21.18 % a 76.12 % para $k = 4$. Esto es consistente con una búsqueda que dispone de más alternativas espaciales para generar y conservar trayectorias factibles.

Las modificaciones realizadas al algoritmo genético buscan reducir algunos de los problemas observados en versiones iniciales. En particular, la mutación reemplaza el gen seleccionado por una acción distinta, evitando mutaciones nulas que dejen al individuo sin cambios. Además, se conserva la mejor ruta factible encontrada durante la evolución y se incorpora un mecanismo de respaldo basado en la memoria de las celdas visitadas cuando el algoritmo no obtiene una ruta factible. Estas medidas buscan reducir la aparición de ciclos y recorridos repetitivos. Sin embargo, no eliminan las limitaciones de la representación: la función de fitness sigue siendo una aproximación estática frente a un entorno cuyo fuego y ocupación cambian durante la simulación.

Las tablas reportan turnos de simulación y no duración de CPU. Por lo tanto, no permiten afirmar que IDA* o el algoritmo genético sean los algoritmos que tardan más en ejecutarse computacionalmente. Durante las pruebas realizadas se observó que ambos podían requerir un mayor tiempo de procesamiento; sin embargo, esta observación no fue medida sistemáticamente y, por lo tanto, no constituye una métrica del benchmark.

Los mecanismos de respaldo y las modificaciones realizadas permitieron mejorar el comportamiento respecto de la versión inicial del algoritmo genético, que presentaba problemas importantes en las tasas de supervivencia. También se observó una complejidad temporal considerable durante algunas ejecuciones de IDA* y del algoritmo genético. No obstante, debido a que estos aspectos no fueron medidos sistemáticamente, los resultados de las versiones anteriores no se incorporaron al análisis cuantitativo presentado en este informe.

7. Limitaciones y mejoras

Una limitación relevante es que no se fija una semilla global. Esto permite representar la naturaleza estocástica del escenario, pero dificulta reproducir exactamente una ejecución. Para futuros experimentos se podría guardar una semilla por corrida y registrar todos los parámetros del fuego.

También se debe registrar explícitamente el valor de p_{pared} y el grosor d utilizado para los cruces de muros, ya que estos parámetros afectan la evolución del entorno.

En IDA*, el límite de 500 nodos constituye una decisión de control experimental. Aumentarlo podría permitir encontrar rutas que actualmente se descartan, pero también incrementaría el costo

computacional de algunas búsquedas. De hecho una de las razones por las que se implementó este límite es porque podían pasar horas completando muy pocas o ninguna iteración.

Para el algoritmo genético, posibles mejoras incluyen aumentar el tamaño de la población, utilizar más generaciones, ajustar la tasa de mutación y diseñar una función de fitness que incorpore de manera más explícita el riesgo dinámico del fuego. También existe el problema de que la zona protegida para la aparición inicial del fuego no funciona como se espera en mapas cuya salida se encuentra fuera de dicha zona. Por lo tanto, una posible mejora sería establecer una zona de protección alrededor de la salida, en lugar de excluir únicamente las últimas filas del mapa.

8. Conclusión

El proyecto implementa los tres paradigmas solicitados: BFS y DFS como búsquedas no informadas, A* e IDA* como búsquedas informadas y un algoritmo genético como método bioinspirado. El sistema integra movimiento ortogonal, espera, capacidad finita, congestión, propagación estocástica del fuego, replanificación y evacuación.

Los resultados muestran que BFS y A* mantienen tasas de supervivencia elevadas en las configuraciones evaluadas. IDA* presenta resultados competitivos en Dispersión Abierta, pero una mayor variabilidad en mapas con restricciones de conectividad. DFS presenta tasas de supervivencia menores y medias de turnos superiores en las configuraciones observadas.

El algoritmo genético muestra una fuerte dependencia de la topología. En particular, su desempeño mejora de forma importante en Dispersión Abierta, donde alcanza 70.58 % con $k = 3$ y 76.12 % con $k = 4$. Las consideraciones como la mutación efectiva, la conservación de la mejor ruta y la memoria de respaldo, permiten evaluar una versión más robusta del método que las implementaciones iniciales.

Finalmente, las conclusiones deben entenderse dentro del protocolo utilizado: 200 simulaciones por combinación, ausencia de una semilla global, propagación ordinaria con $p = 0.75$, cruce de paredes con $p_{\text{pared}} = 0.25$, límite de 500 nodos por búsqueda de IDA* y métricas de turnos obtenidas únicamente en las corridas con tiempo de despeje definido.

9. Declaración de uso de IA generativa y procedencia

Gemini, GitHub Copilot y Codex se utilizaron como herramientas de apoyo durante la implementación y documentación. Después de que el autor creara una estructura base del código, algoritmos como BFS, DFS y A*, estas herramientas se utilizaron como apoyo para revisar, modificar y adaptar componentes del código ante cambios, errores y problemas de integración. También se emplearon para la implementación del algoritmo genético y de IDA*, resolver errores específicos, refinar el código, realizar correcciones relacionadas con la complejidad temporal de IDA* y del algoritmo genético, desarrollar la interfaz gráfica, convertir archivos PNG a texto, realizar la lectura de mapas, refactorizar y optimizar el código, mantener la compatibilidad entre cambios y abordar aspectos más complejos de la gestión de eventos para la propagación del fuego.

También se utilizaron como apoyo en la redacción y revisión del informe, incluyendo la corrección de estilo, la corrección ortotipográfica, la estructuración en LaTeX, la organización de ecuaciones y la maquetación de tablas y figuras.

La integración, revisión y adaptación del código al proyecto fueron realizadas por el autor, así como las decisiones sobre parámetros, estructura de clases, integración e inicialización de agentes, estructura de la grilla, funcionamiento del fuego y configuración experimental.